
Multimodal Hallucination Alignment

Tanush Savadi
COMPSCI 690L: Advanced Topics in Generative AI
University of Massachusetts Amherst

Abstract

This project studies whether image aware preference tuning can make a vision language model less likely to hallucinate. I compare standard DPO with an image aware DPO variant on Qwen2.5-VL-3B-Instruct [3]. The main idea is that a good multimodal model should not only give a fluent answer, but should give an answer that is supported by the image. I evaluate both trained adapters on HallusionBench, POPE, and ChartQA, and I also test each sample with the original image, a blank image, and a mismatched image. The result is mixed but useful. Image aware DPO gives the clearest gain on ChartQA, is roughly tied with standard DPO on HallusionBench, and is slightly worse on POPE. It also changes its answers a bit more often when images are blanked or mismatched, so the hypothesis is partly supported, but the gains are modest.

1 Introduction

Large vision language models can answer questions about images. But they can also hallucinate. In this project, hallucination means the model says something that is not supported by the image. This is a problem because the answer can sound confident even when it is wrong.

The goal of this project is to test whether a model can be made more grounded in the image through preference optimization. I use the word grounded to mean that the model answer depends on the actual image in a useful way.

The project compares two training methods. The first method is standard DPO. The second method is image aware DPO. Both use the same base model and the same training data. The difference is in the training objective.

My main research question is:

Does image aware DPO improve visual grounding and reduce hallucination compared with standard DPO.

My hypothesis was that image aware DPO would help. I expected it to improve performance when the answer depends strongly on the image. I also expected it to make the model more sensitive to image changes.

2 Background

DPO stands for Direct Preference Optimization [1]. It is a way to fine tune a model using preference pairs. Each example has a prompt, a chosen answer, and a rejected answer. The

model learns to prefer the chosen answer.

For a vision language model, the prompt also includes an image. This creates an extra issue. The better answer should be better because it matches the image, not just because it sounds better.

Standard DPO does not directly test this. It can learn that one answer sounds more preferred than another answer. But it may not fully learn whether the answer is supported by the visual input.

The image aware version tries to add this missing pressure. It compares behavior with the correct image and with a mismatched image. The goal is to reward preferences that depend on the right image.

In simple terms:

- standard DPO says, prefer the better answer
- image aware DPO says, prefer the better answer when the image actually supports it

3 System Overview

The project is built as an end to end experiment pipeline. It has four main parts.

- data preparation
- model training
- benchmark evaluation
- dashboard and analysis artifacts

The code prepares data into one shared format. Each example has a sample id, an image path, a prompt, a ground truth answer, metadata, and a mismatched image path. This makes it easier to use the same evaluation code across multiple benchmarks.

The training pipeline creates LoRA adapters for Qwen2.5-VL-3B-Instruct [2, 3]. I do not train the whole model from scratch. This is important because full multimodal training would be too expensive for this class project.

The evaluation pipeline saves predictions and dependence results to Google Drive. This was important because the runs were long and Colab sessions were not always stable. The dashboard reads those saved artifacts. It does not run live model inference.

4 Data

The preference training data comes from `trl-lib/rlaif-v` [4]. The full pilot uses 8000 training examples and 1000 validation examples.

For evaluation I use three benchmarks.

- HallusionBench
- POPE
- ChartQA

HallusionBench is used to test multimodal hallucination [5]. POPE is used for object existence questions [6]. ChartQA is used to test chart based visual question answering [7].

For each evaluation sample, I use three image variants.

- original image
- blank image
- mismatched image

This lets me measure both benchmark accuracy and image dependence. If the model gives the same answer after the image is blanked or swapped, that may mean it is not really using the image. If the answer changes, that means the model is at least sensitive to the image. But changing the answer is not always enough. The changed answer also needs to be useful.

5 Methods

5.1 Standard DPO

The standard DPO model is trained on normal preference pairs. For each image and prompt, the model learns to prefer the chosen response over the rejected response.

This is the baseline alignment method. It tells us what happens when we fine tune the model with preference data but do not add the extra image aware term.

5.2 Image Aware DPO

The image aware DPO model uses the same preference data, but it also considers a mismatched image. The model is encouraged to separate the matched image case from the mismatched image case.

The goal is to make the model less likely to prefer an answer when the image does not support it. This should make the model more visually grounded.

The image aware objective has an extra gap term. This makes training noisier, but it is the part that tries to enforce image dependence.

6 Experiment Setup

The main full pilot setup was:

- base model: Qwen2.5-VL-3B-Instruct
- training examples: 8000
- validation examples: 1000
- optimizer steps: 1000
- gradient accumulation steps: 8
- training method 1: standard DPO
- training method 2: image aware DPO

The two final run ids are:

- 2026-04-08-standard_dpo-pilot-7
- 2026-04-08-image_aware_dpo-pilot-7

All artifacts were saved to Google Drive. This includes adapter weights, metrics, predictions, dependence files, and dashboard parquet files.

7 Engineering Work

A large part of this project was getting the pipeline to run reliably. This was not just a small notebook experiment. The model is large enough that runtime and memory problems matter.

Some important engineering work included:

- setting up Colab GPU training
- saving artifacts to Google Drive
- fixing multimodal batching and image tensor issues
- adding progress logs for long runs
- making evaluation resumable from cached predictions
- fixing POPE yes and no normalization
- fixing a HallusionBench resume bug caused by duplicate sample ids
- building dashboard artifacts for later inspection

The resumable evaluation was important. The full evaluation can take hours. If the runtime disconnects, it should not lose all progress. The final evaluator can reuse cached predictions and only generate missing ones.

The compute side was also a real part of the project. I ran the main workflow through Google Colab over several weeks, rather than as one clean local script. The larger pilot runs needed stronger Colab GPUs such as G4, A100, and H100 class runtimes when they were available. Even with those GPUs, the full pipeline was time consuming. Training both adapters took hours, and full evaluation was slower because each benchmark sample was tested with the original image, a blank image, and a mismatched image. That is why the final system saves every important artifact to Google Drive. Without this, a runtime disconnect or laptop shutdown could lose several hours of work.

This also affected the project design. I did not make the dashboard run live model inference because that would be too heavy for a normal laptop. Instead, the dashboard is artifact driven. It reads the completed Colab outputs and turns them into the final analysis. This made the project more reliable and easier to present.

There was also a POPE scoring issue. The model often answered in full sentences like “Yes, there is a snowboard in the image”. The first scorer expected only exact **yes** or **no**. That made early POPE results look wrong. I fixed this by normalizing common yes and no answer patterns.

There was also a HallusionBench issue. Some HallusionBench rows shared the same sample id. The resume logic was using a cache key that was too simple. It skipped many image aware HallusionBench rows by mistake. I fixed the cache key to include the prompt, ground truth, image path, metadata, and image variant. After that fix, the image aware run had the full expected prediction count.

8 Training Results

Both full pilot training runs completed.

Model	Steps	Last loss	Last DPO loss	Gap loss
standard_dpo	1000	0.4772	0.4772	–
image_aware_dpo	1000	0.9165	0.7508	0.4220

The standard DPO loss is lower, but this should not be over interpreted. The two models are trained with different objectives. The image aware model has extra terms, so its loss is not directly comparable to the standard DPO loss.

The main point is that both training runs completed and produced usable LoRA adapters. The image aware gap loss also shows that the image aware part of the objective was active.

9 Final Evaluation Results

The final evaluation used the corrected prediction files. The image aware run had the expected count:

Benchmark	Original predictions
HallusionBench	1129
POPE	9000
ChartQA	1920

The total prediction count was 36147. This is the sum of all benchmark samples times three image variants.

9.1 Headline Metrics

Benchmark	Metric	standard_dpo	image_aware_dpo
ChartQA	relaxed accuracy	0.3797	0.4063
HallusionBench	accuracy	0.5722	0.5740
POPE	accuracy	0.8733	0.8718
POPE	F1	0.8830	0.8814

These results show a mixed outcome. Image aware DPO is better on ChartQA. It is also slightly better on HallusionBench. Standard DPO is slightly better on POPE.

9.2 ChartQA

ChartQA is the clearest positive result for image aware DPO.

Metric	standard_dpo	image_aware_dpo
augmented relaxed accuracy	0.4990	0.5292
human relaxed accuracy	0.2604	0.2833
overall relaxed accuracy	0.3797	0.4063

The image aware model improves both the augmented split and the human split. This suggests that the image aware objective helped on chart based visual question answering.

9.3 HallusionBench

HallusionBench is almost tied, with a very small advantage for image aware DPO.

Metric	standard_dpo	image_aware_dpo
overall accuracy	0.5722	0.5740
category VD accuracy	0.5550	0.5550
category VS accuracy	0.5911	0.5948

This should not be described as a large improvement. It is better to say that the two models are basically competitive on HallusionBench, with image aware DPO slightly higher.

9.4 POPE

On POPE, standard DPO is slightly higher.

Metric	standard_dpo	image_aware_dpo
accuracy	0.8733	0.8718
precision	0.8650	0.8624
recall	0.9018	0.9014
F1	0.8830	0.8814
yes ratio	0.5202	0.5216

The gap is very small. So I would not say that standard DPO strongly wins on POPE. I would say image aware DPO did not improve POPE in this pilot.

10 Statistical Evidence

After getting the final metrics, I added an artifact only statistical analysis step. This step does not rerun the model. It reads the saved prediction files and compares the two models on the exact same samples. It also uses bootstrap resampling to estimate confidence intervals for the metric differences.

Benchmark	Metric	Delta	95% low	95% high	P(IA > Std)
ChartQA	relaxed accuracy	0.0266	0.0177	0.0359	1.0000
HallusionBench	accuracy	0.0018	-0.0080	0.0124	0.6230
POPE	accuracy	-0.0016	-0.0032	0.0003	0.0410
POPE	F1	-0.0016	-0.0033	0.0000	0.0260

The delta is image aware DPO minus standard DPO. This makes the interpretation more careful. The ChartQA gain looks like the strongest result because its interval is fully above zero. HallusionBench is better described as a near tie because its interval crosses zero. POPE slightly favors standard DPO, but the gap is still very small.

I also counted paired wins and losses. On ChartQA, image aware DPO is correct when standard DPO is wrong on 64 examples, while standard DPO is correct when image aware DPO is wrong on 13 examples. On HallusionBench, the counts are 16 image aware wins and 14 standard wins. On POPE, the counts are 27 image aware wins and 41 standard wins. This supports the same story as the metrics. The image aware method helped most on ChartQA, was basically tied on HallusionBench, and did not improve POPE.

11 Dependence Results

The dependence metrics measure how much the model changes when the image is blanked or mismatched.

Metric	standard_dpo	image_aware_dpo
blank changed rate	0.9667	0.9705
mismatch changed rate	0.9285	0.9361
blank score drop mean	0.4272	0.4200
mismatch score drop mean	0.3508	0.3508

The image aware model changes its answers slightly more often when the image is blanked or mismatched. This supports the idea that it is a bit more sensitive to visual changes.

But the score drop numbers are not clearly better. The blank score drop is slightly lower for image aware DPO. The mismatch score drop is the same. So the image aware model is more

responsive, but this does not translate into a large robustness gain.

12 Dashboard

The project also includes a Streamlit dashboard. The dashboard reads saved run artifacts. It lets me inspect benchmark metrics, statistical evidence, prediction examples, training preferences, failure tags, training curves, and dependence behavior.

The dashboard is useful because the final result is not just one number. Some examples show correct image grounding. Other examples show fluent but wrong answers. Looking at examples helps explain what the metrics mean.

I also added a Story Map page for the final presentation. This page is meant to show the main project story quickly. It includes a method and benchmark network, a Sankey style flow from preference data to model results, a heatmap of metric differences, a dependence tree, and a radar style dependence fingerprint. These views make the result easier to explain because they show where image aware DPO helped and where it did not.

I also added an Evidence page. This page is more like the proof page. It shows bootstrap confidence intervals, paired win and loss counts, bootstrap delta distributions, dependence evidence, and representative examples. This matters because the final result is subtle. The Evidence page helps separate the stronger ChartQA result from the weaker HallusionBench and POPE differences.

The dashboard does not do live model inference on my laptop. That is intentional. The model is too large to run comfortably on my local machine. The heavy model training and evaluation were done on Colab. The local dashboard is for analysis, presentation, and qualitative inspection of the saved results.

The dashboard uses saved files like:

- `metrics.json`
- `predictions.jsonl`
- `dependence.jsonl`
- `dashboard_examples.parquet`
- `dashboard_summary.parquet`
- `evidence_summary.parquet`
- `evidence_paired_cases.parquet`

For the final demo, I would start with the Story Map page, then show the Evidence page, then show a few examples from the Dependence and Examples pages. This gives both the high level result and the concrete evidence behind it.

13 Discussion

The results partly support the hypothesis. Image aware DPO improves ChartQA and is roughly tied with standard DPO on HallusionBench. It also makes the model slightly more likely to change its answer when the image changes.

However, the effect is not large. The POPE result is slightly worse for image aware DPO. The dependence score drops are also not clearly better. So the image aware objective does not dominate standard DPO.

My interpretation is that image aware DPO changed the model in the intended direction, but only modestly. It helped most on ChartQA. It did not give a clear overall win across every hallucination benchmark.

This is still a useful result. It shows that the method has potential, but it also shows that simply adding an image mismatch term is not enough to solve multimodal hallucination.

14 Limitations

There are several limitations.

First, this is a pilot scale experiment. The training set is not huge. I also only ran one final seed. So the results should not be treated as a large scale final claim. They are better understood as a careful pilot study.

Second, the project depends on Colab GPU access. The full pipeline is slow even on stronger Colab GPUs. This made debugging and rerunning expensive. It also means the project is not easy to reproduce quickly on a normal laptop. In practice, this meant the project required careful scheduling around available Colab runtimes and repeated checks that files were being saved to Drive. Some of the engineering work, like checkpointed predictions and cached dashboard files, came directly from this compute constraint.

Third, the model is only one base model. The result might be different for a larger Qwen model or a different model family.

Fourth, the image aware objective is simple. It only uses mismatched images in one way. A stronger method might need better negative images, better losses, or benchmark specific tuning.

Fifth, automatic metrics do not capture everything. The dashboard and qualitative examples are still important for understanding the model behavior.

15 Conclusion

This project built and tested an end to end multimodal alignment pipeline. It prepared data, trained two LoRA adapters, evaluated both models, saved artifacts, and built dashboard data.

The final result is that image aware DPO gives modest grounding related gains. It improves ChartQA from 0.3797 to 0.4063. It is roughly tied on HallusionBench, with 0.5740 compared with 0.5722 for standard DPO. It is slightly worse on POPE, with 0.8718 accuracy compared with 0.8733 for standard DPO.

The dependence analysis also shows that image aware DPO is slightly more sensitive to blank and mismatched images. This means the model is reacting to the image more often. But the improvement is small, and it does not solve hallucination by itself.

So my final answer to the research question is:

Image aware DPO helps in some ways, especially on ChartQA, but it is not a complete solution. It gives a small grounding benefit while staying close to standard DPO on the other benchmarks.

For a class project, I think this is a meaningful result. The system works end to end, the comparison is real, and the final conclusion is honest rather than over claimed.

16 Work Contribution and Code Availability

This was an individual final project, so all project work was completed by me. My contribution includes the project design, data preparation code, training pipeline, image aware DPO implementation, benchmark evaluation, resumable prediction caching, metric fixes, statistical analysis, dashboard, final report, and presentation materials.

The public GitHub repository is:

<https://github.com/tanushsavadi/cs-6901-mm-align>

The repository includes the source code, configs, tests, dashboard code, report source, and final analysis artifacts. The largest raw image datasets and model checkpoints are not all stored directly in the repository because they are too large for normal GitHub usage. Instead, the code is written so the experiment can read those files from local paths or Google Drive paths when available.

References

- [1] Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. Direct Preference Optimization: Your Language Model is Secretly a Reward Model. arXiv:2305.18290, 2023. <https://arxiv.org/abs/2305.18290>
- [2] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-Rank Adaptation of Large Language Models. arXiv:2106.09685, 2021. <https://arxiv.org/abs/2106.09685>
- [3] Shuai Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Sibor Song, Kai Dang, Peng Wang, Shijie Wang, Jun Tang, and others. Qwen2.5-VL Technical Report. arXiv:2502.13923, 2025. <https://arxiv.org/abs/2502.13923>
- [4] TRL contributors. `trl-lib/rlaif-v`: a multimodal preference dataset. Hugging Face Datasets, accessed May 2026. <https://huggingface.co/datasets/trl-lib/rlaif-v>
- [5] Tianrui Guan, Fuxiao Liu, Xiyang Wu, Ruiqi Xian, Zongxia Li, Xiaoyu Liu, Xijun Wang, Lichang Chen, Furong Huang, Yaser Yacoob, Dinesh Manocha, and Tianyi Zhou. HallusionBench: An Advanced Diagnostic Suite for Entangled Language Hallucination and Visual Illusion in Large Vision-Language Models. arXiv:2310.14566, accepted to CVPR 2024. <https://arxiv.org/abs/2310.14566>
- [6] Yifan Li, Yifan Du, Kun Zhou, Jinpeng Wang, Xin Zhao, and Ji-Rong Wen. Evaluating Object Hallucination in Large Vision-Language Models. In Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, pages 292–305, 2023. doi:10.18653/v1/2023.emnlp-main.20. <https://aclanthology.org/2023.emnlp-main.20/>
- [7] Ahmed Masry, Do Xuan Long, Jia Qing Tan, Shafiq Joty, and Enamul Hoque. ChartQA: A Benchmark for Question Answering about Charts with Visual and Logical Reasoning. In Findings of the Association for Computational Linguistics: ACL 2022, pages 2263–2279, 2022. doi:10.18653/v1/2022.findings-acl.177. <https://aclanthology.org/2022.findings-acl.177/>